Mathematical Foundations and Derivations for CK-MPM and Sand Simulation

A learning companion to the paper-to-code map

Purpose. Start with derivatives and small matrices, then follow how geometry, continuum mechanics and discretization produce the equations used in CK-MPM. This is a connected learning route for the paper's core numerical method and the current sand model, not a claim to prove every theorem or GPU optimization in the paper. The companion *Paper-to-Code Guide* locates implementation functions. Equations numbered here belong to this document; "paper Eq." refers to Liu, Wang and Li's CK-MPM paper [1].

The examples and derivations below use ordinary column-vector notation. A result derived from stated assumptions, a model choice and an expression copied from the author implementation are different kinds of evidence. Sections 13 and 17 identify places where that distinction matters. No simulation code is changed by this guide.

Contents

1 Dependency map and notation **3**

2 Derivatives, matrices and the Jacobian **3**

3 Deformation and the velocity gradient **4**

4 Determinants, volume and mass **5**

5 Oriented area and Nanson's formula **5**

6 Traction, Cauchy stress and Piola stress **6**

7 Momentum balance and the weak internal force **6**

8 From integrals to MPM particles and grid nodes **7**

9 Time integration, stability and the two clocks **7**

10 Shape functions and the compact kernel **8**

11 Why two shifted grids are needed **9**

12 Deriving the affine least-squares fit **9**

12.1 Where angular information is stored 10

13 MLS reconstruction and a convention audit **10**

13.1 Keep the derived expression and source expression distinct 11

14 SVD, rotation and logarithmic stretch **11**

15 Deriving elastic stress from Hencky energy **12**

15.1 Why E and Poisson ratio lead to the Lamé parameters 13

16 Elastic/plastic deformation and the sand yield cone **13**

16.1 Pressure-dependent resistance to shear 13

16.2 Radial correction at compressive trace 14

17 The implemented volume-history correction **14**

18 Trace the dependencies through one substep **15**

18.1 A worked symbolic path to the weak nodal force 16

18.2 Questions you should now be able to answer 16

19 Primary references and further study **16**

1 Dependency map and notation

The boxes are clickable reading routes. The table gives the cross-branch prerequisites that a simple chain cannot show.

Derivatives/Jacobian → Deformation → Volume and mass

Area/Nanson → Stress conversion → Weak form → MPM

Kernel → Dual-grid moments → Affine fit → MLS forces

SVD → Elastic stress → Yield/return map → One timestep

Result	Immediate prerequisites
Deformation F	Derivative, Jacobian, first-order Taylor approximation.
Volume J and density	F, determinant, change of variables, conservation of mass.
Nanson's formula	F, oriented area/cross product, determinant and inverse transpose.
Piola stress P	Cauchy traction, equality of physical force, Nanson's formula.
Weak internal force	Momentum balance, integration by parts, P, chain rule.
Particle/grid MPM force	Weak form, shape functions, particle quadrature and lumped mass.
Deformation evolution	Chain rule in space and time; velocity gradient.
CK first-order consistency	Compact weights, partition of unity, two grid offsets.
APIC affine fit	Outer products, weighted least squares, dual-grid moment identities.
MLS force approximation	Affine fit, shape gradients, the weak internal force.
Hencky stress	SVD, logarithmic stretch, differentiating elastic energy.
Sand return mapping	Hencky stress, volumetric/deviatoric split and a yield criterion.

Notation. $\mathbf{X}$ is a material/reference position and $\mathbf{x}$ a current/spatial position. Roman i, j index spatial components, capital A, B index reference components; repeated component indices are summed. A particle index p and a grid index k are not summed unless a summation is written. ∇f is a column gradient, $(\nabla \mathbf{v})_{ij} = \partial v_i/\partial x_j$, and $(\operatorname{Div} \mathbf{P})_i = \partial P_{iA}/\partial X_A$.

Until plasticity is introduced, F is the Jacobian of the full deformation map. In the plasticity sections we distinguish total F from elastic $\mathbf{F}^e$. A corrected elastic matrix stored by a solver must not automatically be treated as the full geometric deformation Jacobian.

2 Derivatives, matrices and the Jacobian

Builds on: Functions, coordinates, addition and multiplication.

A derivative describes how an output changes under a small input change:

$$f'(a) = \lim_{h \to 0} \frac{f(a+h) - f(a)}{h}, \qquad f(a+h) = f(a) + f'(a)h + O(h^2)$$

for a sufficiently smooth function. For $f(x_1, x_2)$, the partial derivative $\partial f/\partial x_1$ varies x_1 while holding x_2 fixed. Collecting the partial derivatives gives the gradient, so a small vector increment obeys $f(\mathbf{x}+\mathbf{h}) \approx f(\mathbf{x}) + \nabla f(\mathbf{x})^T \mathbf{h}$.

For a vector-valued function $\mathbf{f} : \mathbb{R}^n \to \mathbb{R}^m$, each output has its own gradient. Stacking those gradients as rows gives its *Jacobian matrix*:

$$\mathbf{J_f} = \left[\frac{\partial f_i}{\partial x_j}\right], \qquad \mathbf{f}(\mathbf{x}+\mathbf{h}) \approx \mathbf{f}(\mathbf{x}) + \mathbf{J_f}(\mathbf{x})\mathbf{h}. \tag{2.1}$$

For $\mathbf{f}(X_1, X_2) = (2X_1 + X_2, X_2^2)$, the Jacobian at $(1, 1)$ is $\left[\begin{smallmatrix} 2 & 1 \\ 0 & 2 \end{smallmatrix}\right]$. Thus $(.01, 0)$ changes the output by approximately $(.02, 0)$, whereas $(0, .01)$ changes it by approximately $(.01, .02)$.

A transpose exchanges rows and columns. For column vectors $\mathbf{a}, \mathbf{b}$, $\mathbf{a}^T\mathbf{b}$ is a scalar dot product, while $\mathbf{a}\mathbf{b}^T$ is an outer product with entry $a_i b_j$. For matrices, define the double contraction $\mathbf{A} : \mathbf{B} = \sum_{ij} A_{ij} B_{ij} = \mathrm{tr}(\mathbf{A}^T\mathbf{B})$. Matrix order matters: generally $\mathbf{AB} \neq \mathbf{BA}$.

The chain rule is matrix multiplication: $\mathbf{J_{g \circ f}} = \mathbf{J_g J_f}$. An inverse matrix reverses an invertible local map. It does not mean taking the reciprocal of every matrix entry.

Check your understanding. The Jacobian is a matrix; its determinant is a scalar. The symbol J later denotes the determinant of F, not the Jacobian matrix itself.

3 Deformation and the velocity gradient

Builds on: Section 2.

A motion map sends each material point to its current position:

$$\mathbf{x} = \boldsymbol{\phi}(\mathbf{X}, t), \qquad F_{iA} = \frac{\partial \phi_i}{\partial X_A}.$$

Applying the Jacobian approximation to neighboring material points gives

$$\mathrm{d}\mathbf{x} = \mathbf{F}\,\mathrm{d}\mathbf{X}. \tag{3.1}$$

F records local stretching, shearing and rotation. Translation disappears under differentiation. For displacement $\mathbf{u} = \boldsymbol{\phi} - \mathbf{X}$, $\mathbf{F} = \mathbf{I} + \nabla_X \mathbf{u}$.

Examples: a translation has $\mathbf{F} = \mathbf{I}$; a rigid rotation has $\mathbf{F} = \mathbf{R}$, $\mathbf{R}^T\mathbf{R} = \mathbf{I}$; and a uniform stretch has $\mathbf{F} = \mathrm{diag}(s_1, s_2, s_3)$. The shear map $x_1 = X_1 + \gamma X_2$, $x_2 = X_2$ has $\mathbf{F} = \left[\begin{smallmatrix} 1 & \gamma \\ 0 & 1 \end{smallmatrix}\right]$.

Let $\mathbf{v}(\mathbf{x}, t) = \partial_t \boldsymbol{\phi}(\mathbf{X}, t)$ evaluated along that motion. Differentiating F and using the chain rule gives

$$\dot{F}_{iA} = \frac{\partial v_i}{\partial x_j} F_{jA}, \qquad \dot{\mathbf{F}} = \mathbf{L}\mathbf{F}, \quad \mathbf{L} = \nabla_x \mathbf{v}. \tag{3.2}$$

Forward Euler over a small step gives $\mathbf{F}^{\mathrm{trial}} = (\mathbf{I} + \Delta t\, \mathbf{L})\mathbf{F}^n$. This is the deformation-update structure in paper Eq. (19). Later the affine fit approximates L.

For a scalar spatial function $w(\mathbf{x})$, differentiation of $w(\boldsymbol{\phi}(\mathbf{X}))$ gives another useful chain rule:

$$\nabla_X w = \mathbf{F}^T \nabla_x w. \tag{3.3}$$

Check your understanding. Velocity has units length/time, L has units 1/time, and F is dimensionless. Therefore $\mathbf{I} + \Delta t\, \mathbf{L}$ has consistent units.

4 Determinants, volume and mass

Builds on: Section 3; dot and cross products.

Three short edge vectors $\mathbf{a}, \mathbf{b}, \mathbf{c}$ span an oriented volume $\mathbf{a} \cdot (\mathbf{b} \times \mathbf{c})$. A matrix transforms that triple product by its determinant:

$$(\mathbf{Fa}) \cdot [(\mathbf{Fb}) \times (\mathbf{Fc})] = \det(\mathbf{F})\, \mathbf{a} \cdot (\mathbf{b} \times \mathbf{c}).$$

Writing $J = \det \mathbf{F}$, an orientation-preserving deformation has $J > 0$ and

$$\mathrm{d}v = J\, \mathrm{d}V. \tag{4.1}$$

Lowercase $\mathrm{d}v$ is current volume; uppercase $\mathrm{d}V$ is reference volume. With $\mathbf{F} = \mathrm{diag}(2, 1, .5)$, J is one: volume is unchanged despite changes in shape. A rotation has J one too.

A material element keeps its mass. If ρ_R is reference mass density and ρ is current density, then

$$\rho\, \mathrm{d}v = \rho_R\, \mathrm{d}V \quad \Longrightarrow \quad \rho J = \rho_R. \tag{4.2}$$

This is the reference/current mass-balance connection behind the paper's continuum preliminaries. Reference quadrature volume $V_{p,0}$ stays fixed; current geometric volume would be $J_p V_{p,0}$.

For an invertible F, differentiation of a determinant yields $\dot{J} = J \operatorname{tr}(\mathbf{F}^{-1}\dot{\mathbf{F}})$. Using (3.2) and cyclic invariance of trace gives $\dot{J} = J \operatorname{tr} \mathbf{L} = J \nabla \cdot \mathbf{v}$. Differentiating (4.2) then gives $\dot{\rho} + \rho \nabla \cdot \mathbf{v} = 0$.

Check your understanding. J describes local volume change, not a particle count and not the simulation timestep. These meanings must remain separate.

5 Oriented area and Nanson's formula

Builds on: Sections 3 and 4.

A surface patch is spanned by two small tangent vectors $\mathbf{a}, \mathbf{b}$. Their cross product contains both orientation and area: $\mathbf{a} \times \mathbf{b} = \mathbf{N}\, \mathrm{d}A$, where N is the reference unit normal. After deformation, the tangents become F a and F b, so $(\mathbf{Fa}) \times (\mathbf{Fb}) = \mathbf{n}\, \mathrm{d}a$.

To find the transformation, test the deformed area vector against an arbitrary third vector $\mathbf{Fc}$. The determinant identity gives

$$[(\mathbf{Fa}) \times (\mathbf{Fb})] \cdot \mathbf{Fc} = J(\mathbf{a} \times \mathbf{b}) \cdot \mathbf{c},$$
$$\mathbf{F}^T[(\mathbf{Fa}) \times (\mathbf{Fb})] = J(\mathbf{a} \times \mathbf{b}).$$

The second equality follows because the first holds for every c. Multiplying by $\mathbf{F}^{-T}$ proves the area-vector transformation:

$$\boxed{\mathbf{n}\, \mathrm{d}a = J\mathbf{F}^{-T}\mathbf{N}\, \mathrm{d}A = \mathrm{cof}(\mathbf{F})\mathbf{N}\, \mathrm{d}A.} \tag{5.1}$$

Here $\mathbf{F}^{-T} = (\mathbf{F}^{-1})^T$ and $\mathrm{cof}(\mathbf{F}) = J\mathbf{F}^{-T}$ for invertible F. This is Nanson's formula; continuum-mechanics references give this bridge explicitly [2, 3].

Taking magnitude and direction separately gives

$$\frac{\mathrm{d}a}{\mathrm{d}A} = J\|\mathbf{F}^{-T}\mathbf{N}\|, \qquad \mathbf{n} = \frac{\mathbf{F}^{-T}\mathbf{N}}{\|\mathbf{F}^{-T}\mathbf{N}\|}.$$

Worked check. With $\mathbf{F} = \mathrm{diag}(2, 1, .5)$ and $\mathbf{N} = \mathbf{e}_1$, J is one but the patch area scales by $1/2$. Its two tangents stretch by one and $1/2$. A normal along $\mathbf{e}_3$ gives area factor two instead.

Check your understanding. Why not multiply surface area by J alone? Different orientations stretch differently. J transforms volume; Nanson transforms oriented area.

6 Traction, Cauchy stress and Piola stress

Builds on: Section 5.

Traction is force per unit current area on a plane with unit normal n. The Cauchy stress tensor $\boldsymbol{\sigma}$ maps that normal to traction: $\mathbf{t} = \boldsymbol{\sigma}\mathbf{n}$. The physical differential force must be identical when described per reference area. Define first Piola stress P by $\mathrm{d}\mathbf{f} = \mathbf{P}\mathbf{N}\,\mathrm{d}A$. Insert Nanson's formula into the current-area description:

$$\mathrm{d}\mathbf{f} = \boldsymbol{\sigma}\mathbf{n}\,\mathrm{d}a = J\boldsymbol{\sigma}\mathbf{F}^{-T}\mathbf{N}\,\mathrm{d}A.$$

Since this holds for every reference normal,

$$\boxed{\mathbf{P} = J\boldsymbol{\sigma}\mathbf{F}^{-T}}, \qquad \boxed{\mathbf{T} = \mathbf{P}\mathbf{F}^{T} = J\boldsymbol{\sigma}}. \tag{6.1}$$

T is the Kirchhoff stress in this notation. Sandia calls its Cauchy stress T; we use $\boldsymbol{\sigma}$ to avoid confusing it with the stored $\mathbf{P}\mathbf{F}^T$ in our shader. A symbol's name is not its physical definition. See also the stress-measure teaching reference [4].

Under ordinary balance without couple stresses, $\boldsymbol{\sigma}$ and T are symmetric. P need not be symmetric because its input normal is reference-based and its output force is spatial. It is a two-configuration tensor.

Worked check. If $\mathbf{F} = 2\mathbf{I}$, then $J = 8$ and $\mathbf{P} = 4\boldsymbol{\sigma}$. A reference patch grows to four times its area, so this correctly preserves its physical force.

Check your understanding. Nanson's formula explains the conversion of stress measures. It is not an extra sand force or an algorithm called once per particle.

7 Momentum balance and the weak internal force

Builds on: Sections 6 and 3; integration.

For a material region, Newton's law equates momentum change to body and surface forces. Applying the divergence theorem to the surface traction gives the reference strong form

$$\rho_R\ddot{\boldsymbol{\phi}} = \mathrm{Div}_X\,\mathbf{P} + \rho_R\mathbf{g}. \tag{7.1}$$

The divergence theorem here means integrating a derivative inside a region is equivalent to integrating its normal flux on the boundary. For our row-divergence convention, $\int_{\Omega_0}\mathrm{Div}_X\,\mathbf{P}\,\mathrm{d}V = \int_{\partial\Omega_0}\mathbf{P}\mathbf{N}\,\mathrm{d}A$.

Multiply (7.1) by a scalar test/shape function $w_i(\mathbf{x})$ and integrate. Component-wise integration by parts gives

$$\begin{aligned}\int_{\Omega_0}\rho_R w_i\ddot{\phi}_a\,\mathrm{d}V = &-\int_{\Omega_0}P_{aA}\frac{\partial w_i}{\partial X_A}\,\mathrm{d}V \\ &+\int_{\partial\Omega_0}w_i(PN)_a\,\mathrm{d}A + \int_{\Omega_0}\rho_R w_i g_a\,\mathrm{d}V.\end{aligned} \tag{7.2}$$

Here i labels a grid shape function, while a labels a vector component: they are independent indices. The minus sign is the integration-by-parts sign. For an interior node or zero prescribed traction, the explicit boundary term vanishes. Boundary conditions otherwise contribute force or constraints.

Use (3.3) to move the shape gradient into spatial coordinates:

$$\mathbf{f}_i^{\mathrm{int}} = -\int_{\Omega_0}\mathbf{P}\mathbf{F}^T\nabla_x w_i\,\mathrm{d}V = -\int_{\Omega_0}\mathbf{T}\nabla_x w_i\,\mathrm{d}V. \tag{7.3}$$

This explains both the negative sign and the $\mathbf{P}\mathbf{F}^T$ in paper Eq. (2). The factor F transpose is supplied by the chain rule; Nanson supplied the stress conversion earlier in the dependency chain.

There is a second check using current volume: $\boldsymbol{\sigma}\,\mathrm{d}v = \boldsymbol{\sigma} J\,\mathrm{d}V = \mathbf{T}\,\mathrm{d}V$. Thus $-\int_{\Omega_t} \boldsymbol{\sigma}\nabla w_i\,\mathrm{d}v$ is the same force.

Check your understanding. Do not multiply by J a second time when already using reference volume and $\mathbf{T} = \mathbf{P}\mathbf{F}^T$. That would count the volume conversion twice.

8 From integrals to MPM particles and grid nodes

Builds on: Section 7; weighted sums.

Particle quadrature approximates a reference-volume integral by $\int_{\Omega_0} q\,\mathrm{d}V \approx \sum_p V_{p,0} q_p$. Particles carry mass $m_p = \rho_{R,p} V_{p,0}$ and state history; grid nodes provide locations for collecting and updating motion.

Interpolate a grid field by shape functions $w_{i,p} = N_i(\mathbf{x}_p)$. The consistent mass matrix is approximated by $M_{ij} \approx \sum_p m_p w_{i,p} w_{j,p}$. With partition of unity $\sum_j w_{j,p} = 1$, row-summing gives a diagonal *lumped* nodal mass $m_i = \sum_p m_p w_{i,p}$. This replaces a coupled mass solve with independent nodal updates.

For a single grid, the elementary transfers and force are

$$m_i = \sum_p w_{i,p} m_p, \qquad \mathbf{q}_i = \sum_p w_{i,p} m_p \mathbf{v}_p, \tag{8.1}$$

$$\mathbf{f}_i^{\mathrm{int}} = -\sum_p V_{p,0} \mathbf{T}_p \nabla w_{i,p}, \qquad \mathbf{u}_i = \frac{\mathbf{q}_i + \Delta t\, \mathbf{f}_i^{\mathrm{int}}}{m_i} + \Delta t\, \mathbf{g}. \tag{8.2}$$

The divisions apply only to sufficiently massive active nodes. Particles then gather grid velocity and advect using the new velocity. In a traditional kernel-gradient formulation, $\mathbf{L}_p \approx \sum_i \mathbf{u}_i (\nabla w_{i,p})^T$ feeds (3.2). The affine formulation below supplies another estimate.

In our code, P2G adds stress momentum directly to q, so the grid update only normalizes the already-forced momentum, adds gravity and handles boundaries. This rearranges stages; it does not justify dropping the stress force. For CK, each grid gets its own full mass/momentum scatter, and the gathering and physical conservation totals account for averaging the two grids.

Tiny transfer example. Two equal-mass particles send weight 1/2 to the same node with velocities +1 and −1. That node has positive mass and zero momentum, hence zero velocity. Repeating weighted averaging can erase velocity variation. Particle count is not the same thing as independent grain contacts: this is a discretized continuum.

Check your understanding. Forces have units mass times length/time squared. In $V_{p,0}\mathbf{T}\nabla w$, volume contributes length cubed, stress force/area and gradient inverse length, giving force.

9 Time integration, stability and the two clocks

Builds on: Sections 3 and 8.

The particle update $\mathbf{x}^{n+1} = \mathbf{x}^n + \Delta t\, \mathbf{v}^{n+1}$ uses the updated velocity. Together with an explicit velocity kick this is the symplectic-Euler pattern. It does not allow an arbitrary timestep.

For a spring $\ddot{x} = -\omega^2 x$, the same pattern gives

$$\begin{pmatrix} x^{n+1} \\ v^{n+1} \end{pmatrix} = \begin{pmatrix} 1 - h^2\omega^2 & h \\ -h\omega^2 & 1 \end{pmatrix} \begin{pmatrix} x^n \\ v^n \end{pmatrix}.$$

The matrix has determinant one and trace $2 - h^2\omega^2$. Oscillatory bounded modes require $0 < h\omega < 2$; at the endpoint the repeated eigenvalue can be defective. Larger steps can amplify errors even when the intended motion is slow. Elastic waves give a related scale $h \sim \Delta x/c$.

For small isotropic elastic disturbances, define

$$\mu = \frac{E}{2(1+\nu)}, \qquad \lambda = \frac{E\nu}{(1+\nu)(1-2\nu)}, \qquad c = \sqrt{\frac{\lambda + 2\mu}{\rho}}.$$

Our automatic acoustic estimate is $\Delta t = \mathrm{CFL}\,\Delta x/c$. It is a practical estimate, not a proof of nonlinear elastoplastic stability.

A numerical step advances simulated time by Δt regardless of how long the computer takes. With s steps per frame and R rendered frames per real second, the observed time ratio is approximately $s\Delta t R$. For $s = 3$, $\Delta t = .0004$ and $R = 60$, the ratio is .072. Larger s adds work and may reduce R; larger Δt changes integration accuracy and stability. These are different controls.

Check your understanding. A 16 ms computer work budget is not automatically 16 ms of stable physical evolution. Deriving $\Delta t =$ desired simulated frame time$/s$ is possible, but its numerical stability must be checked separately.

10 Shape functions and the compact kernel

Builds on: Sections 2 and 8.

A shape function is a position-dependent interpolation weight. Useful properties include locality, nonnegativity and partition of unity. It is not simply the Euclidean distance; distance is an input to its formula.

The 1D linear tent is $K_{\mathrm{lin}}(r) = \max(1 - |r|, 0)$. Its slope jumps at zero and at the support edges. A force that uses its derivative can therefore change abruptly when a particle crosses a cell boundary.

The CK paper's Eq. (4) chooses

$$K(r) = \begin{cases} 1 - |r| + \dfrac{\sin(2\pi|r|)}{2\pi}, & |r| \le 1, \\ 0, & |r| > 1. \end{cases} \tag{10.1}$$

One can motivate the coefficient without assuming the result. Start on $0 < r < 1$ with $1 - r + a\sin(2\pi r)$. Requiring the derivative to vanish at zero gives $-1 + 2\pi a = 0$, hence $a = 1/(2\pi)$. This is the simple chosen correction; it is not a proof that no other smooth compact kernels exist.

For $0 < r < 1$, differentiation gives

$$K'(r) = -1 + \cos(2\pi r), \qquad K''(r) = -2\pi \sin(2\pi r).$$

The first and second derivatives approach zero at the join points. Together with even symmetry and the zero exterior, this makes K C^2 continuous. Since $K' \le 0$ on $(0, 1)$ and $K(0) = 1$, $K(1) = 0$, weights are nonnegative. The sine integrates to zero over $(0, 1)$, so symmetry gives $\int_{\mathbb{R}} K(r)\,\mathrm{d}r = 1$.

For $d \in [0, 1]$,

$$K(d) + K(1 - d) = 1, \tag{10.2}$$

because $\sin(2\pi(1 - d)) = -\sin(2\pi d)$. This gives the two cell weights $w_0 = K(d)$, $w_1 = 1 - w_0$ in `shaders/interpolation.glsl`. In 3D, multiply the three axis weights; each grid contributes $2^3 = 8$ nodes. The paper's Eq. (5) is this tensor product.

Check your understanding. At $d = .25$, $w_0 = .75 + 1/(2\pi) \approx .9092$ and $w_1 \approx .0908$. They sum to one but are not the linear weights .75 and .25.

11 Why two shifted grids are needed

Builds on: Section 10; weighted averages and outer products.

Partition of unity reproduces constants. Reproducing a linear position field also needs the first moment to agree with particle position. On one grid, write $x_p/\Delta x = b + d$, where b is the left node coordinate and $0 \le d < 1$. The weighted node coordinate is

$$bw_0 + (b + 1)w_1 = b + d - \frac{\sin(2\pi d)}{2\pi}.$$

The last term is an error in linear reproduction, even though the weights sum to one.

Use grids shifted by $-\Delta x/4$ and $+\Delta x/4$. Their local d values differ by half a cell modulo one. Since $\sin(2\pi(d + 1/2)) = -\sin(2\pi d)$, averaging the two first moments cancels the sine errors. Tensor-product weights extend this argument to each component in 3D. This is the key consistency role of the dual-grid construction in paper §4.2.

Define the combined weighted average

$$\langle a \rangle_p = \frac{1}{2} \sum_{k \in \{-1,+1\}} \sum_i w_{i,p,k} a_{i,k}, \qquad \mathbf{r}_{i,p,k} = \mathbf{x}_{i,k} - \mathbf{x}_p.$$

For a complete interior support,

$$\langle 1 \rangle_p = 1, \qquad \langle \mathbf{r} \rangle_p = \mathbf{0}, \qquad \mathbf{D}_p = \langle \mathbf{r}\mathbf{r}^T \rangle_p. \tag{11.1}$$

For any vector z, $\mathbf{z}^T \mathbf{D} \mathbf{z} = \langle (\mathbf{z} \cdot \mathbf{r})^2 \rangle \ge 0$. Thus D is positive semidefinite, and it is invertible if the positively weighted offsets span all spatial directions. It has units length squared.

Each grid receives the full scattered mass. Summing both grid masses would count material twice; a physical total uses half their sum. Likewise the velocity and moment reconstruction use the combined average above.

Boundary clipping can invalidate the complete-support identities. Our GPU checks therefore require transfer stencils to stay inside the grid. Applying a wall constraint to velocity also exerts a physical boundary impulse; momentum need not remain constant when gravity or walls act.

Check your understanding. Sixteen grid nodes are not sixteen new sand grains. They are two sets of grid samples supporting the same material particle.

12 Deriving the affine least-squares fit

Builds on: Sections 2 and 11.

Approximate a nearby velocity field by $\mathbf{v}(\mathbf{x}_p + \mathbf{r}) \approx \mathbf{v}_p + \mathbf{C}_p \mathbf{r}$. C describes spatial variation: a skew-symmetric C is rotation, a symmetric part describes stretching/shearing. APIC carries this variation through particle/grid transfer [5].

Choose v and C to minimize the combined weighted residual

$$E(\mathbf{v}, \mathbf{C}) = \tfrac{1}{2}\langle \|\mathbf{u}_i - \mathbf{v} - \mathbf{C}\mathbf{r}_i\|^2 \rangle. \tag{12.1}$$

Differentiating with respect to v gives $\mathbf{v} = \langle \mathbf{u} \rangle - \mathbf{C}\langle \mathbf{r} \rangle = \langle \mathbf{u} \rangle$. Differentiating entry C_{ab} gives

$$0 = -\langle (u_a - v_a - C_{aj} r_j) r_b \rangle.$$

Using the zero first moment leaves the normal equations

$$\mathbf{CD} = \mathbf{B}, \qquad \mathbf{B} = \langle \mathbf{u}\mathbf{r}^T \rangle, \qquad \boxed{\mathbf{C} = \mathbf{B}\mathbf{D}^{-1}}. \tag{12.2}$$

These are the dual-grid D and B behind paper §4.4 and Eqs. (20)–(21). B has units length squared/time, so C has units inverse time.

G2P uses offsets $\widehat{\mathbf{r}} = \mathbf{r}/\Delta x$ in cell units. With $\widehat{\mathbf{B}} = \sum w \mathbf{u} \widehat{\mathbf{r}}^T$ and $\widehat{\mathbf{D}} = \sum w \widehat{\mathbf{r}} \widehat{\mathbf{r}}^T$, it therefore computes

$$\mathbf{C} = \frac{1}{\Delta x} \widehat{\mathbf{B}} \widehat{\mathbf{D}}^{-1}.$$

The omitted halves cancel. Forgetting the division by Δx would change the physical units and the amount of deformation.

P2G then transfers $w m_p(\mathbf{v}_p + \mathbf{C}_p \mathbf{r}_i)$. For an exact affine field $\mathbf{u}_i = \mathbf{a} + \mathbf{A}\mathbf{x}_i$, the moment identities give $\mathbf{v}_p = \mathbf{a} + \mathbf{A}\mathbf{x}_p$ and $\mathbf{C}_p = \mathbf{A}$. This supplies an analytic test independent of another implementation. For rigid rotation around z, $\mathbf{A} = \left[\begin{smallmatrix} 0 & -\omega & 0 \\ \omega & 0 & 0 \\ 0 & 0 & 0 \end{smallmatrix}\right]$. The fitted field correctly sends opposite sides in opposite directions.

12.1 Where angular information is stored

The grid's angular momentum contribution of one affine particle expands to

$$\langle (\mathbf{x}_p + \mathbf{r}) \times m_p(\mathbf{v}_p + \mathbf{C}_p \mathbf{r}) \rangle = \mathbf{x}_p \times m_p \mathbf{v}_p + m_p \langle \mathbf{r} \times \mathbf{C}_p \mathbf{r} \rangle.$$

The cross terms vanish by $\langle \mathbf{r} \rangle = 0$. The second term is local spin information absent from a point-position/point-velocity pair. It depends on $\mathbf{CD} = \mathbf{B}$. Measuring APIC angular momentum using only $\sum_p \mathbf{x}_p \times m_p \mathbf{v}_p$ omits this part. This expansion explains the mechanism, not a full floating-point conservation proof for our force/boundary/plasticity pipeline.

Check your understanding. For a constant velocity field, B is zero because the first moment is zero, hence C is zero. For an affine field, the fit should recover C exactly in exact arithmetic with a complete support.

13 MLS reconstruction and a convention audit

Builds on: Sections 7 and 12.

For a scalar field, fit $a_i \approx a_0 + \mathbf{q}^T \mathbf{r}_i$ using the same weighted objective. The normal equations give $a_0 = \langle a \rangle$ and $\mathbf{q} = \mathbf{D}^{-1}\langle \mathbf{r} a \rangle$. Reconstruct at a nearby point z while holding the fit center $\mathbf{x}_p$ fixed:

$$a(\mathbf{z}) \approx \tfrac{1}{2} \sum_{k,i} N_{i,k}(\mathbf{z}) a_{i,k}, \tag{13.1}$$

$$N_{i,k}(\mathbf{z}) = w_{i,p,k}[1 + (\mathbf{z} - \mathbf{x}_p)^T \mathbf{D}_p^{-1} \mathbf{r}_{i,p,k}], \tag{13.2}$$

$$\nabla_z N_{i,k}(\mathbf{x}_p) = w_{i,p,k} \mathbf{D}_p^{-1} \mathbf{r}_{i,p,k}. \tag{13.3}$$

The weights and moment matrix are frozen while differentiating in z. Moving the fit center and differentiating its weights is a different operation. The block moment matrix for basis $(1, \mathbf{r})$ is $\left[\begin{smallmatrix} 1 & 0 \\ 0 & \mathbf{D} \end{smallmatrix}\right]$ because of (11.1). This is the concrete linear-basis connection to the general MLS discussion in paper §4.5 [6].

Substituting (13.3) into (7.3) and using particle quadrature gives, in the column-vector convention of this guide,

$$\boxed{\mathbf{f}_{i,k}^{\text{int}} = -\sum_p V_{p,0} w_{i,p,k} \mathbf{T}_p \mathbf{D}_p^{-1} \mathbf{r}_{i,p,k}.} \tag{13.4}$$

For dual-grid nodal variables, the half in the physical averaged reconstruction appears in both nodal mass and force. It cancels in the update when using the code's full per-grid mass storage. It does not change matrix multiplication order.

13.1 Keep the derived expression and source expression distinct

The current shader and the inspected author CUDA source instead form

$$\mathbf{A}_{\text{source}} = m_p \mathbf{C}_p - \Delta t \, V_{p,0} \mathbf{D}_p^{-1} \mathbf{T}_p, \qquad \mathbf{q}_{i,k} \mathrel{+}= w(m_p \mathbf{v}_p + \mathbf{A}_{\text{source}} \mathbf{r}).$$

The source puts $\mathbf{D}^{-1}$ on the left of T, while (13.4) places it on the right. These matrices are generally not interchangeable. For instance, in a 2D block,

$$\mathbf{D} = \begin{pmatrix} 2 & 0 \\ 0 & 1 \end{pmatrix}, \quad \mathbf{T} = \begin{pmatrix} 1 & 1 \\ 1 & 2 \end{pmatrix}, \quad \mathbf{D}^{-1}\mathbf{T} = \begin{pmatrix} .5 & .5 \\ 1 & 2 \end{pmatrix} \neq \mathbf{T}\mathbf{D}^{-1} = \begin{pmatrix} .5 & 1 \\ .5 & 2 \end{pmatrix}.$$

Symmetry of both factors does not make their product commute. The distinction can disappear for a scalar D, such as the interior quadratic B-spline moment $\mathbf{D} = (\Delta x^2/4)\mathbf{I}$, but CK moments need not be scalar matrices.

Audit boundary. The paper-to-code document records what the code actually does. This derivation records what follows under our stated column-vector/gradient conventions. Resolving their difference needs an explicit tensor-index and force-path audit; agreement between references that share the source ordering is insufficient. This guide does not silently swap the code's factors or claim that the discrepancy is already resolved.

Another independent distinction is paper Eq. (17): its printed velocity transfer includes grid-mass/particle-mass factors, whereas our shader follows the author's CUDA average $\mathbf{v}_p = \langle \mathbf{u} \rangle$. That source comparison is not a derivation of equality to the printed equation.

Check your understanding. The chain from Nanson through weak force to MLS is now complete. Which multiplication order does that chain give, and which does the source use? Keeping both answers visible is part of understanding the method.

14 SVD, rotation and logarithmic stretch

Builds on: Sections 2 and 3.

For an invertible deformation with positive determinant, $\mathbf{F}^T\mathbf{F}$ is symmetric positive definite. Its orthonormal eigenvectors form V and its positive eigenvalues are s_a^2. Define $\mathbf{S} = \text{diag}(s_1, s_2, s_3)$ and $\mathbf{U} = \mathbf{F}\mathbf{V}\mathbf{S}^{-1}$. Then

$$\mathbf{U}^T\mathbf{U} = \mathbf{S}^{-1}\mathbf{V}^T\mathbf{F}^T\mathbf{F}\mathbf{V}\mathbf{S}^{-1} = \mathbf{I}, \qquad \boxed{\mathbf{F} = \mathbf{U}\mathbf{S}\mathbf{V}^T}.$$

This constructs the singular value decomposition from the spectral theorem for symmetric matrices. The values s_a are principal stretches, not entries of F. The polar rotation is $\mathbf{R} = \mathbf{U}\mathbf{V}^T$ and the right stretch is $\mathbf{V}\mathbf{S}\mathbf{V}^T$. A rigid rotation has all $s_a = 1$.

For sequential one-dimensional stretches, multiplying stretch factors should add strain increments: $e(ab) = e(a) + e(b)$. With continuity and the small-strain normalization $e'(1) = 1$, this motivates $e(s) = \log s$. Therefore define principal logarithmic strains $\epsilon_a = \log s_a$. Their trace is $\sum_a \epsilon_a = \log \prod_a s_a = \log J$ when F is the total positive-determinant deformation.

Split them into mean and shape-changing parts:

$$t = \epsilon_1 + \epsilon_2 + \epsilon_3, \quad \overline{\boldsymbol{\epsilon}} = \boldsymbol{\epsilon} - \tfrac{t}{3}\mathbf{1}, \quad \sum_a \overline{\epsilon}_a = 0. \tag{14.1}$$

The vector $\mathbf{1} = (1, 1, 1)^T$ here is not the identity matrix. The trace describes volume; differences among the strains describe distortion.

Worked check. For $\mathbf{F} = \mathrm{diag}(2, 1, .5)$, $\boldsymbol{\epsilon} = (\log 2, 0, -\log 2)$ and $t = 0$. It is volume-preserving but not strain-free. Setting strain equal to J alone would lose its shear energy.

Check your understanding. SVD is used to expose stretches for the material model. It is not a separate particle-grid interaction rule.

15 Deriving elastic stress from Hencky energy

Builds on: Sections 6 and 14.

A constitutive model specifies how deformation produces stress; MPM itself does not determine this law. For the elastic stretch energy used in the sand model, write an isotropic Hencky energy density

$$\Psi(\mathbf{F}) = \mu \sum_a \epsilon_a^2 + \frac{\lambda}{2}\left(\sum_a \epsilon_a\right)^2. \tag{15.1}$$

Stress is the energy derivative defined by $\delta\Psi = \mathbf{P} : \delta\mathbf{F}$. For a singular-value energy, the derivative has the principal form $\mathbf{P} = \mathbf{U}\,\mathrm{diag}(\partial\Psi/\partial s_a)\mathbf{V}^T$. Since $\partial\epsilon_a/\partial s_a = 1/s_a$, ordinary differentiation gives

$$\frac{\partial\Psi}{\partial s_a} = \frac{2\mu\epsilon_a + \lambda t}{s_a}, \tag{15.2}$$

$$\mathbf{P} = \mathbf{U}\,\mathrm{diag}\left(\frac{2\mu\epsilon_a + \lambda t}{s_a}\right)\mathbf{V}^T, \tag{15.3}$$

$$\mathbf{T} = \mathbf{P}\mathbf{F}^T = \mathbf{U}\,\mathrm{diag}(2\mu\epsilon_a + \lambda t)\mathbf{U}^T. \tag{15.4}$$

For distinct stretches the principal derivative follows by differentiating $\mathbf{F} = \mathbf{U}\mathbf{S}\mathbf{V}^T$: rotational variations do not change an isotropic energy, leaving its diagonal stretch variation. Repeated positive stretches are interpreted by continuity; this is not permission to differentiate a particular numerical eigenvector sign convention.

These equations explain the `pHat`, U, V and final multiplication by `transpose(F)` in `sandMaterial()`. With $s_a = 1$, every logarithm is zero and the stress is zero even for a rigid rotation.

15.1 Why E and Poisson ratio lead to the Lamé parameters

For small strains the same law linearizes to $\boldsymbol{\sigma} = 2\mu\boldsymbol{\varepsilon} + \lambda\,\mathrm{tr}(\boldsymbol{\varepsilon})\mathbf{I}$. In uniaxial tension, take strains (a, b, b) and zero lateral stresses. Then $2\mu b + \lambda(a + 2b) = 0$, so

$$\nu = -b/a = \frac{\lambda}{2(\lambda+\mu)}, \qquad E = \sigma_{11}/a = \frac{\mu(3\lambda+2\mu)}{\lambda+\mu}.$$

Solving these two relations gives

$$\mu = \frac{E}{2(1+\nu)}, \qquad \lambda = \frac{E\nu}{(1+\nu)(1-2\nu)}.$$

These expressions are used in Section 9 and in `parameters()` in `src/simulation_config.cpp`.

Check your understanding. Doubling E at fixed Poisson ratio doubles both elastic moduli; wave speed grows approximately by $\sqrt{2}$, reducing the acoustic step scale.

16 Elastic/plastic deformation and the sand yield cone

Builds on: Sections 14 and 15.

Elastic deformation can recover; plastic deformation changes the stress-free configuration. A common multiplicative description is

$$\mathbf{F}^{\text{total}} = \mathbf{F}^e\mathbf{F}^p.$$

A trial update first holds plastic history fixed and evolves the elastic part: $\mathbf{F}^{e,\text{trial}} = (\mathbf{I}+\Delta t\,\mathbf{L})\mathbf{F}^{e,n}$. Return mapping then changes elastic stretches to satisfy a yield criterion. This is why the code updates F in G2P and corrects it in the material pass.

The stored corrected matrix is an elastic state. Its determinant is not necessarily the total geometric J used in Nanson's formula and density conversion. The full plastic matrix need not be stored if the chosen stress update only needs the elastic state and selected history variables.

Under the specific assumption that reference energy is $\Psi(\mathbf{F}^e)$ with $\mathbf{F}^p$ held fixed during elastic differentiation,

$$\delta\mathbf{F}^e = \delta\mathbf{F}^{\text{total}}(\mathbf{F}^p)^{-1}, \qquad \mathbf{P}^{\text{total}} = \mathbf{P}^e(\mathbf{F}^p)^{-T}, \qquad \mathbf{P}^{\text{total}}(\mathbf{F}^{\text{total}})^T = \mathbf{P}^e(\mathbf{F}^e)^T.$$

This explains how an elastic-stretch stress calculation can supply the Kirchhoff-like product used by the force path. Different plastic energy/volume conventions require their own constitutive derivation.

16.1 Pressure-dependent resistance to shear

Use tensile stress as positive. The compression-positive pressure is $p = -\,\mathrm{tr}\,\mathbf{T}/3$, and the deviatoric stress is $\mathbf{T}_{\text{dev}} = \mathbf{T} - (\mathrm{tr}\,\mathbf{T}/3)\mathbf{I}$. A Drucker–Prager cone in the normalization used here is

$$f(\mathbf{T}) = \|\mathbf{T}_{\text{dev}}\|_F + \alpha\,\mathrm{tr}\,\mathbf{T} \le 0. \tag{16.1}$$

Equivalently the allowed shear norm is at most $3\alpha p$: compression raises shear resistance. At zero pressure a zero-cohesion cone supports no deviatoric stress. The implementation uses the friction-angle calibration

$$\alpha = \sqrt{\frac{2}{3}}\,\frac{2\sin\varphi}{3-\sin\varphi}.$$

This calibration is a material-model convention, not a consequence of the MPM transfer kernel. Friction-angle calibrations vary with the chosen invariant normalization and fitting to a friction model. An internal material angle is not a universal sliding threshold for every object or boundary.

From (15.4), the stress eigenvalues are $2\mu\epsilon_a + \lambda t$. Thus $\|\mathbf{T}_{\text{dev}}\|_F = 2\mu\|\bar{\boldsymbol{\epsilon}}\|$ and $\operatorname{tr}\mathbf{T} = (2\mu + 3\lambda)t$. Dividing the yield function by 2μ gives

$$\Delta\gamma = \|\bar{\boldsymbol{\epsilon}}\| + \frac{3\lambda + 2\mu}{2\mu}\alpha t. \tag{16.2}$$

This derives the scalar expression seen in the return mapping when the auxiliary volume-history correction is zero.

16.2 Radial correction at compressive trace

For $t < 0$ and $\Delta\gamma > 0$, use a non-associated, trace-preserving plastic flow in the deviatoric direction:

$$\boldsymbol{\epsilon}^{\text{new}} = \boldsymbol{\epsilon} - \frac{\Delta\gamma}{\|\bar{\boldsymbol{\epsilon}}\|}\bar{\boldsymbol{\epsilon}}. \tag{16.3}$$

The trace stays t. The new deviatoric norm is $\|\bar{\boldsymbol{\epsilon}}\| - \Delta\gamma = -(3\lambda+2\mu)\alpha t/(2\mu)$, which is nonnegative under compression. Substitution into (16.2) gives zero: the returned state lies on the cone. If $\Delta\gamma \le 0$, the trial state is already admissible. For tensile/noncompressive trace, the source's zero-cohesion apex treatment sets elastic logarithmic stretches to zero. Then reconstruct

$$s_a^{\text{new}} = \exp(\epsilon_a^{\text{new}}), \qquad \mathbf{F}^{e,\text{new}} = \mathbf{U}\,\operatorname{diag}(s_a^{\text{new}})\mathbf{V}^T.$$

Zero logarithmic stretch gives a rotation $\mathbf{U}\mathbf{V}^T$, not necessarily an identity matrix. The general sand-model background is Klár et al. [7].

Check your understanding. Return mapping is a projection of material state, not the G2P transfer. The transfer provides a trial deformation; plasticity decides which part of that deformation can remain elastic.

17 The implemented volume-history correction

Builds on: Section 16; reading a branch condition.

The CK-MPM material overload adds a scalar history state named `logJp`. The shader uses this specific algorithm, with h denoting the old history:

$$\epsilon_a = \log\max(|s_a|, 10^{-4}), \qquad s = \sum_a \epsilon_a, \qquad t_c = s + h,$$

$$\mathbf{e} = \boldsymbol{\epsilon} - \tfrac{t_c}{3}\mathbf{1}, \qquad q = \|\mathbf{e}\|, \qquad k = \frac{3\lambda + 2\mu}{2\mu}.$$

If $t_c \ge 0$, it sets $s_a^{\text{new}} = 1$ and updates $h^{\text{new}} = h + s$. If $t_c < 0$, it resets $h^{\text{new}} = 0$ and computes

$$\Delta\gamma = q + k\alpha t_c, \qquad \mathbf{H} = \begin{cases} \boldsymbol{\epsilon} - (\Delta\gamma/\max(q, 10^{-20}))\mathbf{e}, & \Delta\gamma > 0, \\ \boldsymbol{\epsilon}, & \Delta\gamma \le 0, \end{cases} \qquad s_a^{\text{new}} = e^{H_a}.$$

The corrected elastic matrix and its stress are then stored.

Important distinction. For nonzero old h, $\sum_a e_a = -h$, so e is not the ordinary traceless deviator from (14.1). The radial-return proof above establishes the zero-history case; it must not be presented as a proof of every volume-correction branch. This extension is traced directly to the

author's ../CK-MPM/include/mpm_material.cuh and implemented in shaders/material.glsl. A full physical derivation of that particular history rule requires the material-model conventions beyond the CK paper's kernel derivation. The guide states the algorithm rather than inventing one.

Likewise, singular-value floors and denominator floors are numerical guards, not material laws. GPU finiteness checks do not prove positive total J, physical accuracy, constitutive admissibility in every corner case or stability of an arbitrary manually selected timestep.

Check your understanding. When h is zero, e is the ordinary deviator and the cone derivation applies. When h is nonzero, identify exactly which quantity changed before reusing that proof.

18 Trace the dependencies through one substep

Builds on: All preceding branches; use the dependency map to revisit a missing link.

The runtime dispatches

$$\text{clear} \rightarrow \text{P2G} \rightarrow \text{grid update} \rightarrow \text{G2P} \rightarrow \text{material update}.$$

At the beginning of a step, each particle has position, velocity, elastic F, affine C, material stress and history. Corrected stress from the previous step feeds the current force scatter. The initial F is identity with zero stress.

Stage	Mathematical ancestry and implementation
Weights	Derivative smoothness → compact sine kernel → dual-grid moment consistency; shaders/interpolation.glsl.
P2G	Particle quadrature plus affine least squares supplies mass and local momentum. Nanson → Piola stress → weak force → moment-based force approximation supplies the stress term, subject to Section 13.1; shaders/p2g.comp.
Grid update	Lumped mass → independent velocity normalization; explicit integration adds gravity and wall constraints; shaders/grid_update.comp.
G2P	Dual-grid moment identities → B and D → fitted C; chain rule $\dot{F} = LF$ → elastic trial update; updated velocity → advection; shaders/g2p.comp.
Material	SVD → logarithmic stretch → Hencky energy derivative → yield cone and return correction → stored stress/history; shaders/material_update.comp, shaders/material.glsl.
Checks	Numerical diagnostics summarize finite state, domain/stencil validity and maximum speed; shaders/particle_health.comp.

18.1 A worked symbolic path to the weak nodal force

The complete bridge that often gets compressed into one line is

$$\begin{aligned}
\mathbf{x} = \boldsymbol{\phi}(\mathbf{X}) &\Rightarrow \mathbf{F} = \nabla_X \boldsymbol{\phi},\ J = \det \mathbf{F}, \\
(\mathbf{Fa}) \times (\mathbf{Fb}) &\Rightarrow \mathbf{n}\,\mathrm{d}a = J\mathbf{F}^{-T}\mathbf{N}\,\mathrm{d}A, \\
\mathrm{d}\mathbf{f} = \boldsymbol{\sigma}\mathbf{n}\,\mathrm{d}a &\Rightarrow \mathbf{P} = J\boldsymbol{\sigma}\mathbf{F}^{-T}, \\
\rho_R \ddot{\boldsymbol{\phi}} = \operatorname{Div}\mathbf{P} + \rho_R\mathbf{g} &\Rightarrow \mathbf{f}_i^{\text{int}} = -\int \mathbf{P}\nabla_X w_i\,\mathrm{d}V, \\
\nabla_X w_i = \mathbf{F}^T\nabla_x w_i &\Rightarrow \mathbf{f}_i^{\text{int}} = -\int \mathbf{P}\mathbf{F}^T\nabla_x w_i\,\mathrm{d}V, \\
\int q\,\mathrm{d}V \approx \sum_p V_{p,0} q_p &\Rightarrow \mathbf{f}_i^{\text{int}} \approx -\sum_p V_{p,0}\mathbf{P}_p\mathbf{F}_p^T\nabla_x w_{i,p}.
\end{aligned}$$

Every arrow now has a derivation above. CK changes the interpolation and dual-grid consistency; affine/MLS constructions change the transfer and gradient approximation. A constitutive law supplies P. These are connected but distinct pieces of the method.

18.2 Questions you should now be able to answer

1. Why are F, J, P and $\boldsymbol{\sigma}$ different mathematical objects?
2. Why does surface area require $J\mathbf{F}^{-T}$ rather than J alone?
3. Which two transformations explain $\mathbf{P}\mathbf{F}^T$ in the weak force?
4. Why do the compact weights need two shifted grids for linear reproduction?
5. Why is $\mathbf{C} = \mathbf{B}\mathbf{D}^{-1}$ a fit rather than an arbitrary extra term?
6. Which factors cancel when code accumulates unaveraged cell-unit moments?
7. Which matrix ordering follows from the conventional MLS force derivation, and which appears in the inspected source?
8. Why does changing dt alter stability, while changing step count primarily changes work per rendered frame?
9. Why does sand need both an elastic stress law and a plastic correction?
10. Why is a finite returned state not itself a proof of physical correctness?

19 Primary references and further study

The displayed calculations are explanatory derivations under stated assumptions. For the missing continuum bridge, start with the Nanson/stress references; for material-model details beyond the zero-history proof, follow the sand paper and the exact author overload cited above. The project reference tests check implementation behavior; they do not replace these mathematical arguments.

References

[1] M. Liu, X. Wang and M. Li. *CK-MPM: A Compact-Kernel Material Point Method*. ACM TOG 44(4), 2025. doi:10.1145/3731155. Local copy: `../../../CK-MPM Paper.pdf`.

[2] Sandia National Laboratories. *Sierra/SM Theory Manual*, Chapter 7, Stress Measures, especially Nanson's Formula and First Piola–Kirchhoff Stress. https://www.sandia.gov/files/sierra/SM_Theory_5_18/main/stress_measures.html.

[3] NPTEL, Continuum Mechanics, Module 3, Lecture 19. *Area and Volume Transformation*. https://archive.nptel.ac.in/content/storage2/courses/112103167/module3/lec19.pdf.

[4] University of Alberta, Engineering Courses. *First and Second Piola Kirchhoff Stress Tensors*. https://engcourses-uofa.ca/books/introduction-to-solid-mechanics/stress/first-and-second-piola-kirchhoff-stress-tensors/.

[5] C. Jiang et al. *The Affine Particle-In-Cell Method*. ACM TOG, 2015. doi:10.1145/2766996.

[6] Y. Hu et al. *A Moving Least Squares Material Point Method with Displacement Discontinuity and Two-Way Rigid Body Coupling*. ACM TOG, 2018. doi:10.1145/3197517.3201293.

[7] G. Klár et al. *Drucker–Prager Elastoplasticity for Sand Animation*. ACM TOG, 2016. doi:10.1145/2897824.2925906.